

Elastic Speculative Slots:

A Formally Verified Protocol for In-Order Retirement and Natural Streaming of Concurrent Blocks in a Terminal Viewport

August 24, 2026

Abstract

Interactive terminal applications increasingly multiplex *speculative* output blocks—streaming model tokens, concurrent build steps, live spinners—inside a bounded live viewport, while the terminal’s own scrollbar demands an immutable, append-only commitment discipline. We present *Elastic Speculative Slots*, a rendering protocol that reconciles these constraints through a decoupled three-layer contract: mutable semantic block state, an exactly-once logical history ledger, and the physical native scrollbar of the host terminal. The protocol admits blocks in order, presents them in elastically allocated slots under a strict reservation invariant with graceful overflow summarization and zero-height degradation, streams append-only blocks line-by-line under row pressure, retires finalized blocks in order under pressure or flush, and survives width/height resizes via three policies (**Preserve**, **Append**, **Rebuild**) with a bottom-first, single-write synchronous replay that is tearing-free by construction. Partial terminal write failures are fail-stop: the native scrollbar receives a prefix of immutable rows and the commit frontier never advances, precluding duplicated or reordered retirement. We formalize the protocol as a guarded transition system, prove a global inductive safety theorem (exact ledger content, capacity exactness, overflow forcing, final-snapshot immutability, epoch-scoped native monotonicity, resize logical-neutrality, fail-stop safety), prove physical realizability on a standard scrolling terminal via a generic streaming lemma, and establish conditional progress theorems under weak fairness, including a precise delineation of when pressure-driven retirement is guaranteed. Every definition and transition corresponds one-to-one to the mechanically checked TLA⁺ specification `ElasticSlots.tla`; TLC exhaustively verified ten invariants and ten temporal properties over three boundary configurations totaling 272,860 distinct states with no violations.

Contents

1	Introduction	1
2	System Model and Formal Definitions	3
2.1	Preliminaries	3
2.2	Parameters and alphabets	3
2.3	Protocol state	4
2.4	Presentation contracts	4
2.5	The two histories	6
2.6	Live geometry	6
2.7	Replay geometry	7

3	The Rendering Protocol as a Transition System	8
3.1	Admission and speculation	8
3.2	Elastic allocation	8
3.3	Finalization	9
3.4	Natural streaming of the append-only head	9
3.5	Retirement	9
3.6	Resize and synchronous bottom-first replay	10
3.7	Shutdown	11
3.8	Scheduler gate and fairness	11
3.9	Conformance summary	12
4	Formal Theorems and Proofs	12
4.1	Invariant catalogue	12
4.2	Exact capacity and overflow bounds	14
4.3	Reservation dominance and bridge termination	15
4.4	Stable-head emission and ledger monotonicity	15
4.5	Generic terminal streaming realization	16
4.6	Bottom-first synchronous replay	17
4.7	Global inductive safety	18
4.8	Conditional progress and liveness	22
5	Mechanical Verification in TLA⁺	23
5.1	Transition correspondence	23
5.2	Checked invariants and temporal properties	23
5.3	Configurations and results	25
6	Conclusion	25

1 Introduction

A terminal emulator is a stateful streaming device. Its scrollbar is a line-buffered, append-only log: once a row scrolls off the top of the visible screen it is frozen in the user’s history and cannot be revised. Modern command-line applications, however, produce output that is intrinsically *speculative* while it is being produced—a language-model response being streamed token by token, a package build whose progress bar rewrites itself, a test runner whose live summary shrinks and grows. Such applications must therefore maintain a bounded *live viewport* in which content may be repainted arbitrarily, and a disciplined boundary across which content is *retired* into scrollbar exactly once, in order, and never again touched.

This paper gives a complete formal treatment of one such discipline, the *Elastic Speculative Slots* protocol, exactly as specified in the TLA⁺ module `ElasticSlots.tla`. The protocol rests on a decoupled three-layer contract:

1. *Semantic application state*. A monotonically ordered family of blocks $1, \dots, N$, each carrying a speculative snapshot (its *want*) and, upon finalization, an immutable *final* snapshot.
2. *Logical history ledger*. A width-independent, exactly-once, in-order sequence of committed semantic rows, extended only at its end, plus the stable emitted prefix of at most one active append-only head block.

3. *Physical native history.* The rows actually scrolled into the terminal’s scrollbar, each tagged with the *proof source* that emitted it (Append, Retire, Replay, Resize, FailedWrite, Exit) and the width at which it was rendered.

The layers are related by invariants, not by identity: the ledger is a pure function of the commit frontier and the final snapshots; the native history is an epoch-scoped, append-only physical realization whose rows are always attributable to committed content, to the stable head prefix, or to explicitly tagged resize/exit/failure events. Resizes may re-render the entire native display (Rebuild) without perturbing the ledger by even one row.

Contributions.

1. A precise mathematical model of the protocol state, its live-viewport geometry (elastic slot allocation, reservation dominance, shrink bridges, overflow summarization, zero-height degradation), and its dual history ledgers (§2).
2. A guarded-command transition system covering the full lifecycle: admission, speculation, allocation, natural append-only streaming, finalization, pressure- and flush-driven retirement, three resize policies with bottom-first synchronous replay, write-failure fail-stop, and graceful and detached shutdown (§3), together with a conformance table of required and forbidden implementation effects.
3. Full proofs of: exact capacity and overflow forcing (Theorem 4.2); reservation dominance and bridge termination (Lemma 4.4); stable-head emission and ledger monotonicity (Lemma 4.6); a generic streaming-realization lemma connecting abstract native appends to concrete VT scroll mechanics (Lemma 4.8); the bottom-first synchronous replay invariant (Lemma 4.11); a global inductive safety theorem (Theorem 4.13); and conditional progress theorems under weak fairness (§4.8), including an explicit counterexample delimiting when pure row pressure does *not* force retirement.
4. A one-to-one mapping from the mathematics to the mechanically checked actions, invariants, and temporal properties of `ElasticSlots.tla`, with exhaustive TLC model-checking results over three boundary configurations (§5).

2 System Model and Formal Definitions

2.1 Preliminaries

$\mathbb{N}$ denotes the naturals including 0; for $k \in \mathbb{N}$ we write $[k] = \{1, \dots, k\}$ (so $[0] = \emptyset$). For a set X , X^* is the set of finite sequences over X ; $\langle \rangle$ is the empty sequence; $|s|$ is the length of s ; $s[j]$ is the j -th element (1-indexed); and

$$s[l..u] = \langle s[l], s[l+1], \dots, s[u] \rangle, \quad \text{with } s[l..u] = \langle \rangle \text{ whenever } l > u.$$

Concatenation is written $s \cdot t$, iterated concatenation $\bigodot_{j=l}^u s_j = s_l \cdot s_{l+1} \cdot \dots \cdot s_u$ (empty when $l > u$), and x^k denotes the sequence consisting of k copies of x . The *prefix order* is

$$s \sqsubseteq t \iff |s| \leq |t| \wedge \forall j \in [|s|] : s[j] = t[j].$$

$\text{last}(s) = s[|s|]$ for nonempty s . All proofs use only these elementary operations.

2.2 Parameters and alphabets

Assumption 2.1 (Protocol parameters). The protocol is parameterized by

- $N \in \mathbb{N} \setminus \{0\}$, the number of block identities, $\mathcal{B} = [N]$;
- $H \in \mathbb{N} \setminus \{0\}$, the maximum transcript (viewport) height;
- $R_{\max} \in \mathbb{N}$, a bound on the number of resize events;
- $\Lambda \in \mathbb{N} \setminus \{0\}$, the uncommitted-block pressure threshold;
- a finite nonempty *row alphabet* Σ and a finite *snapshot universe* $\mathbb{S} \subseteq \Sigma^*$ with $\langle \rangle \in \mathbb{S}$, containing at least one snapshot of length 1 and one of length > 1 ;
- pairwise-distinct symbols $\perp, \sqcup, \square, \boxplus \notin \Sigma$ (respectively: “no final snapshot”, the placeholder row, the blank row, and the overflow-summary row).

We write $L_{\max} = \max\{|s| : s \in \mathbb{S}\}$ and $K_{\max} = 2NL_{\max}$, an upper bound on the physical length of any single write batch (the factor 2 arises from narrow-width rendering, Definition 2.2).

Definition 2.2 (Widths and rendering). The width domain is $\mathbb{W} = \{\text{wide}, \text{narrow}\}$. The *width rendering function* $\rho_{\omega} : \Sigma^* \rightarrow \Sigma^*$ is

$$\rho_{\text{wide}}(s) = s, \quad \rho_{\text{narrow}}(\langle \rangle) = \langle \rangle, \quad \rho_{\text{narrow}}(\langle x \rangle \cdot s) = \langle x, x \rangle \cdot \rho_{\text{narrow}}(s).$$

ρ_{narrow} doubles every semantic row: it is the two-point abstraction of soft-wrap reflow, in which a semantic row may occupy a different number of physical terminal rows at different widths. All that the proofs use is that ρ_{ω} is prefix-monotone and length-homogeneous:

Proposition 2.3. *For all $s, t \in \Sigma^*$ and $\omega \in \mathbb{W}$: (i) $|\rho_{\omega}(s)| = \delta_{\omega}|s|$ with $\delta_{\text{wide}} = 1$, $\delta_{\text{narrow}} = 2$; (ii) $s \sqsubseteq t \Rightarrow \rho_{\omega}(s) \sqsubseteq \rho_{\omega}(t)$; (iii) $\rho_{\omega}(s \cdot t) = \rho_{\omega}(s) \cdot \rho_{\omega}(t)$.*

Proof. Immediate induction on $|s|$ from Definition 2.2. □

Definition 2.4 (Cells, ledger rows, native rows). Let $\Gamma = \Sigma \cup \{\sqcup, \square, \boxplus\}$ be the *cell alphabet* and $\mathbb{Q} = \{\text{Append}, \text{Retire}, \text{Replay}, \text{Resize}, \text{FailedWrite}, \text{Exit}\}$ the *source alphabet*. Then:

- a *cell* is a pair $(o, x) \in (\{0\} \cup \mathcal{B}) \times \Gamma$ (owner 0 marks non-block chrome); the *blank cell* is $\beta = (0, \square)$ and the *overflow cell* is $\omega_{\boxplus} = (0, \boxplus)$;
- a *ledger row* is a pair $(i, x) \in \mathcal{B} \times \Sigma$;
- a *native row* is a quadruple $(q, o, x, \omega) \in \mathbb{Q} \times (\{0\} \cup \mathcal{B}) \times \Gamma \times \mathbb{W}$.

For $i \in \mathcal{B}$ and $s \in \Sigma^*$ define the tagging operators

$$\text{tag}_i(s) = \langle (i, s[1]), \dots, (i, s[|s|]) \rangle,$$

and, with $r = \rho_{\omega}(s)$,

$$\text{ntag}_{\omega}^q(i, s) = \langle (q, i, r[1], \omega), \dots, (q, i, r[|r|], \omega) \rangle,$$

and for a cell sequence u the lift $\text{ntag}_{\omega}^q(u) = \langle (q, o_1, x_1, \omega), \dots \rangle$ where $u[j] = (o_j, x_j)$.

Note the asymmetry, which is fundamental: *ledger* rows are semantic and width-independent; *native* rows are physically rendered at a definite width and carry a provenance source.

2.3 Protocol state

Definition 2.5 (State space). A *protocol state* is a tuple with the components of Table 1. We refer to components by their symbols; a primed symbol denotes the value in the successor state of a transition.

Symbol	TLA ⁺	Type	Meaning
c	c	$0..N$	commit frontier: blocks $1..c$ are committed
φ	phase	$\mathcal{B} \rightarrow \Phi$	lifecycle phase; $\Phi = \{\text{Absent}, \text{Queued}, \text{Active}, \text{Finalized}, \text{Committed}\}$
μ	mode	$\mathcal{B} \rightarrow \mathbb{M}$	presentation contract; $\mathbb{M} = \{\perp, \text{Mutable}, \text{AppendOnly}\}$
w	want	$\mathcal{B} \rightarrow \mathbb{S}$	speculative snapshot
f	final	$\mathcal{B} \rightarrow \mathbb{S} \cup \{\perp\}$	immutable final snapshot
e	emitted	$\mathcal{B} \rightarrow 0..L_{\max}$	stable emitted prefix length (append-only head)
α	alloc	$\mathcal{B} \rightarrow 0..H$	painted slot allocation (rows)
τ	target	$\mathcal{B} \rightarrow 0..H$	requested slot allocation
L	history	$(\mathcal{B} \times \Sigma)^*$	logical history ledger
P	native	native rows*	physical native scrollback (current epoch)
ω	width	$\mathbb{W}$	terminal width class
h	height	$0..H$	live transcript height
ν	resizes	$0..R_{\max}$	resize counter
ε	epoch	$0..R_{\max}$	display epoch
m_r	replayMode	$\{\text{None}, \text{Append}, \text{Rebuild}\}$	pending replay policy
ℓ_r, u_r	replayCursor, replayEnd	$0..N+1, 0..N$	replay window $[\ell_r..u_r]$ over committed blocks
p_r	replayPartial	$0..L_{\max}$	replayed stable-head prefix length
$prep$	replayPrepared	$\mathbb{B}$	replay frame prepared
cut	replayCut	$0..K_{\max}$	rows that must scroll during replay
$flush$	flush	$\mathbb{B}$	explicit retirement request
$shut$	shutdown	$\mathbb{B}$	graceful shutdown initiated
run	running	$\mathbb{B}$	host live
$stop$	stopReason	$\mathbb{T}$	termination cause; $\mathbb{T} = \{\text{Running}, \text{Graceful}, \text{Detach}, \text{WriteFailure}\}$

Table 1: Protocol state components and their counterparts in `ElasticSlots.tla`.

Definition 2.6 (Initial state). The initial state has $c = 0$; $\varphi_i = \text{Absent}$, $\mu_i = \perp$, $w_i = \langle \rangle$, $f_i = \perp$, $e_i = \alpha_i = \tau_i = 0$ for all i ; $L = P = \langle \rangle$; $\omega = \text{wide}$; $h = H$; $\nu = \varepsilon = 0$; $m_r = \text{None}$, $\ell_r = u_r = p_r = cut = 0$, $prep = \text{false}$; $flush = shut = \text{false}$; $run = \text{true}$; $stop = \text{Running}$.

Definition 2.7 (Derived quantities). In any state:

$$\begin{aligned}
 \gamma &= |\{i \in \mathcal{B} : \varphi_i \neq \text{Absent}\}| && \text{(created count)} \\
 U_i(s) &= \begin{cases} s[e_i+1..|s|] & \text{if } \mu_i = \text{AppendOnly} \\ s & \text{otherwise} \end{cases} && \text{(unemitted suffix)} \\
 \text{rep} &\iff m_r \neq \text{None} && \text{(replay pending)}.
 \end{aligned}$$

The lifecycle invariant (Theorem 4.13) guarantees that the created blocks are exactly $[\gamma]$ and that they are partitioned as $1..c$ committed, $c+1..\gamma$ live (queued, active, or finalized), $\gamma+1..N$ absent. We freely use this shape below.

2.4 Presentation contracts

Every created block carries one of two contracts, fixed at creation:

- **Mutable:** while $\varphi_i \in \{\text{Queued}, \text{Active}\}$ the snapshot w_i may be replaced by *any* member of $\mathbb{S}$; the block's viewport slot is repainted in place. No row of a mutable block may reach scrollbar before the whole block retires.
- **AppendOnly:** the snapshot evolves only by prefix extension ($w_i \sqsubseteq w'_i$). Consequently an ever-growing *stable prefix* exists, and while the block is the *head* (the first uncommitted block, $i = c+1$) its stable rows may scroll naturally, line by line, into scrollbar ahead of finalization—this is the streaming affordance.

2.5 The two histories

Definition 2.8 (Committed ledger and stable head). For $k \in 0..N$ define the *committed ledger*

$$C(k) = \bigodot_{j=1}^k \text{tag}_j(f_j).$$

The *partial-head predicate* is

$$\text{ph} \iff c < \gamma \wedge \mu_{c+1} = \text{AppendOnly} \wedge \varphi_{c+1} \in \{\text{Active}, \text{Finalized}\} \wedge e_{c+1} > 0,$$

and the corresponding *active head* prefix is

$$A(c) = \begin{cases} \text{tag}_{c+1}(w_{c+1}[1..e_{c+1}]) & \text{if ph} \\ \langle \rangle & \text{otherwise.} \end{cases}$$

The central ledger invariant, proved inductive in Theorem 4.13, is the *exactness equation*

$$L = C(c) \cdot A(c). \tag{1}$$

The logical history is thus never an independent data structure: it is at all times *exactly* the committed finals in block order followed by the stable emitted prefix of the single append-only head, with no duplication, omission, or reordering. The physical history P , by contrast, is an epoch-scoped append-only realization: within a display epoch it only grows at the end; the Rebuild resize policy resets it to $\langle \rangle$ and increments ε .

2.6 Live geometry

Throughout this subsection fix the ambient tuple $(\varphi, f, e, \omega, h)$; all geometric operators are functions of it (the TLA^+ operators take these as explicit parameters, which is essential for stating transition guards over *successor* values).

Definition 2.9 (Presentation). Block i is *presented*, written $\text{pres}(i)$, iff

$$\varphi_i = \text{Active} \vee (\varphi_i = \text{Finalized} \wedge |\rho_\omega(U_i(f_i))| > 0).$$

Let $\pi = |\{i : \text{pres}(i)\}|$ be the presented count. The viewport *overflows*, written ovf , iff $\pi > h$; the *summary row count* is

$$\sigma = \begin{cases} 1 & \text{if } h > 0 \wedge \text{ovf} \\ 0 & \text{otherwise.} \end{cases}$$

With $\text{newer}(i) = |\{j > i : \text{pres}(j)\}|$, block i is *visible*, written $\text{vis}(i)$, iff

$$\text{pres}(i) \wedge (\text{ovf} \Rightarrow h > 0 \wedge \text{newer}(i) < h - 1).$$

Under overflow, visibility privileges recency: only the $h - 1$ presented blocks with the largest indices remain visible, and one row is sacrificed to a summary marker $\boxplus$ standing for everything older.

Definition 2.10 (Allocation admissibility). Allocation vectors $\alpha, \tau \in (0..H)^{\mathcal{B}}$ are *admissible*, written $\mathcal{A}(\alpha, \tau)$ (with the ambient tuple implicit), iff

1. for every i with $\text{vis}(i)$: if $\varphi_i = \text{Active}$ then $\alpha_i \in 1..H$ and $\tau_i \in 1..H$; otherwise ($\varphi_i = \text{Finalized}$) $\alpha_i \in 1..H$ and $\tau_i = \alpha_i$;
2. for every i with $\neg \text{vis}(i)$: $\alpha_i = \tau_i = 0$;
3. the *reservation invariant* holds:

$$\text{Res}(\alpha, \tau) + \sigma \leq h, \quad \text{where} \quad \text{Res}(\alpha, \tau) = \sum_{i=1}^N \max(\alpha_i, \tau_i). \quad (2)$$

The *canonical allocation* is $\kappa = (i \mapsto \text{if } \text{vis}(i) \text{ then } 1 \text{ else } 0)$.

The reservation invariant charges every block for the *maximum* of its painted and requested heights: growth is admitted only if the grown extent is already reserved, and a shrinking block retains its old reservation until the shrink is actually painted. This is what makes concurrent elastic animation safe (Lemma 4.4).

Definition 2.11 (Row demand and pressure). The *row demand* of block i and the total demand are

$$\text{dm}(i) = \begin{cases} \max(1, |\rho_\omega(U_i(w_i))|) & \varphi_i \in \{\text{Queued}, \text{Active}\} \\ |\rho_\omega(U_i(f_i))| & \varphi_i = \text{Finalized} \\ 0 & \text{otherwise,} \end{cases} \quad D = \sum_{i=1}^N \text{dm}(i).$$

Then *row pressure* is $D > h$; *pressure* is $\text{press} \iff D > h \vee \gamma - c \geq \Lambda$; and *retirement is requested* iff $\text{req} \iff \text{flush} \vee \text{press}$.

Definition 2.12 (Screen layout). The *preview source* and *preview cell* of block i are

$$\text{pv}(i) = \begin{cases} U_i(w_i) & \varphi_i = \text{Active} \\ U_i(f_i) & \text{otherwise,} \end{cases} \quad \text{cell}(i) = \begin{cases} (i, \text{last}(\rho_\omega(\text{pv}(i)))) & \rho_\omega(\text{pv}(i)) \neq \langle \rangle \\ (i, \boxplus) & \text{otherwise.} \end{cases}$$

With $A_\Sigma = \sum_i \alpha_i$, the *screen* is the cell sequence

$$Q = \beta^{h - A_\Sigma - \sigma} \cdot \omega_{\boxplus}^\sigma \cdot \bigodot_{i=1}^N \text{cell}(i)^{\alpha_i}. \quad (3)$$

The model abstracts a slot's visual content to α_i copies of a representative cell carrying the owner and the last rendered unemitted row; a conforming implementation displays the α_i -row tail window of $\rho_\omega(\text{pv}(i))$ (padded with the placeholder when empty). Blank rows sit at the top, then the overflow marker if any, then the slots in ascending block order—the newest block is adjacent to the cursor at the bottom. Well-definedness of (3) (in particular that $\alpha_i > 0$ implies $\text{pv}(i)$ is a genuine snapshot and the exponent of β is nonnegative) is part of Theorem 4.2.

2.7 Replay geometry

Definition 2.13 (Replay frame). When *rep* holds, the *replay frame* is

$$\mathcal{R} = \left(\bigodot_{j=\ell_r}^{u_r} \text{ntag}_{\omega}^{\text{Replay}}(j, f_j) \right) \cdot \text{ntag}_{\omega}^{\text{Replay}}(u_r+1, w_{u_r+1}[1..p_r]),$$

(the second factor being empty when $p_r = 0$), and $\mathcal{R} = \langle \rangle$ otherwise. The *replay room* and *required cut* are

$$\text{room} = |\{j \in [h] : Q[j] = \beta\}|, \quad \text{cut}^* = \max(0, |\mathcal{R}| - \text{room}).$$

When *prep* holds, the *prepared tail* is $\mathcal{R}[\text{cut}+1..|\mathcal{R}|]$.

The replay frame re-renders, at the *current* width, all committed finals in the window $[\ell_r..u_r]$ (invariantly $\ell_r = 1$, $u_r \leq c$) plus the already-emitted stable prefix of the head. The cut cut^* is the exact number of frame rows that cannot fit in the blank region of the viewport and must therefore scroll into native history.

3 The Rendering Protocol as a Transition System

Convention 3.1. Each transition below is a guarded command over the state of Definition 2.5. A transition may fire only when its guard holds; its effect lists the components it changes; *all unmentioned components are unchanged*. Every guard implicitly conjoins *run*. Derived operators (*vis*, σ , κ , $\mathcal{A}$, Q , ...) evaluated in the effect of a transition are evaluated at the *successor* values of their arguments; where a guard constrains a successor configuration we make the substitution explicit. Substitution into a function is written $\varphi[i \mapsto p]$.

3.1 Admission and speculation

CREATE(m), $m \in \{\text{Mutable}, \text{AppendOnly}\}$. *Guard*: $\neg \text{shut} \wedge \gamma < N$. *Effect*:

$$\varphi'_{\gamma+1} := \text{Queued}, \quad \mu'_{\gamma+1} := m.$$

Blocks are created contiguously in index order; the index order is the commitment order.

ADMIT(i). *Guard*: $\neg \text{shut} \wedge \varphi_i = \text{Queued} \wedge \forall j < i : \varphi_j \neq \text{Queued}$, and with $\varphi^\dagger = \varphi[i \mapsto \text{Active}]$, $\alpha^\dagger = \alpha[i \mapsto 1]$, $\tau^\dagger = \tau[i \mapsto 1]$:

$$\neg \text{ovf}(\varphi^\dagger, f, e, \omega, h) \wedge \mathcal{A}(\alpha^\dagger, \tau^\dagger) \text{ at } (\varphi^\dagger, f, e, \omega, h).$$

Effect: $\varphi' := \varphi^\dagger$, $\alpha' := \alpha^\dagger$, $\tau' := \tau^\dagger$. Admission is FIFO (no earlier block is still queued) and *denied* rather than *summarized* when it would overflow the viewport: overflow may arise only from finalization of queued blocks or from *resize*, never from admission.

UPDATE(i, s), $s \in \mathbb{S}$. *Guard*: $\neg \text{shut} \wedge \varphi_i \in \{\text{Queued}, \text{Active}\} \wedge (\mu_i = \text{Mutable} \vee w_i \sqsubseteq s) \wedge s \neq w_i$. *Effect*: $w'_i := s$. This is the only transition by which speculation evolves; the prefix guard is the entirety of the append-only contract.

3.2 Elastic allocation

REQUESTALLOCATION($\tau^\dagger$), $\tau^\dagger \in (0..H)^B$. *Guard:* $\neg shut \wedge \mathcal{A}(\alpha, \tau^\dagger) \wedge \tau^\dagger \neq \tau$. *Effect:* $\tau' := \tau^\dagger$. Targets may change only if the joint reservation (2) of the *current* paint and the *new* request still fits.

Definition 3.2 (Shrink bridge). The *bridge function* $B : (0..H)^2 \rightarrow 0..H$ is

$$B(a, t) = \begin{cases} t & a < t \quad (\text{growth is immediate}) \\ 2 & a > 2 \wedge t = 1 \quad (\text{deep shrink bridges through 2}) \\ t & \text{otherwise.} \end{cases}$$

APPLYALLOCATION(i). *Guard:* $\neg shut \wedge \varphi_i = \text{Active} \wedge \alpha_i \neq \tau_i \wedge \mathcal{A}(\alpha[i \mapsto B(\alpha_i, \tau_i)], \tau)$. *Effect:* $\alpha'_i := B(\alpha_i, \tau_i)$. A slot collapsing from $k > 2$ rows to 1 paints an intermediate 2-row frame, preserving animation stability (the user perceives a contraction, not a snap); Lemma 4.4 shows the bridge terminates and never violates the reservation.

3.3 Finalization

FINALIZEACTIVE(i, s), $s \in \mathbb{S}$. *Guard:* $\neg shut \wedge \varphi_i = \text{Active} \wedge (\mu_i = \text{Mutable} \vee w_i \sqsubseteq s)$. *Effect:*

$$\varphi'_i := \text{Finalized}, \quad w'_i := s, \quad f'_i := s, \quad \alpha' := \tau' := \kappa \text{ at } (\varphi', f', e, \omega, h).$$

FINALIZEQUEUED(i, s), $s \in \mathbb{S}$. Identical, with guard $\varphi_i = \text{Queued}$: a queued block may complete without ever being admitted to a live slot (e.g. a task that finished before any viewport space became available). Finalization freezes the semantic value (f_i is never rewritten; Theorem 4.16) and collapses *all* allocations to canonical single-row slots—finalized content no longer animates.

3.4 Natural streaming of the append-only head

APPENDSTABLE. *Guard:*

$$\neg shut \wedge \neg rep \wedge c < \gamma \wedge \mu_{c+1} = \text{AppendOnly} \wedge \varphi_{c+1} \in \{\text{Active}, \text{Finalized}\} \wedge D > h \wedge e_{c+1} < |w_{c+1}|.$$

Effect: with $n = e_{c+1} + 1$,

$$L' := L \cdot \langle (c+1, w_{c+1}[n]) \rangle, \quad P' := P \cdot \text{ntag}_{\omega}^{\text{Append}}(c+1, w_{c+1}[n..n]), \\ e'_{c+1} := n, \quad \alpha' := \tau' := \kappa \text{ at } (\varphi, f, e', \omega, h).$$

One stable row of the head scrolls naturally into both histories. The transition is enabled only under *row* pressure: while the viewport has room, stable rows stay live (and remain revisable in presentation, though not in value). Only the head block ever streams: this preserves in-order retirement.

COMPLETEAPPENDONLY. *Guard:* $\neg rep \wedge c < \gamma \wedge \mu_{c+1} = \text{AppendOnly} \wedge \varphi_{c+1} = \text{Finalized} \wedge e_{c+1} = |f_{c+1}|$. *Effect:*

$$c' := c + 1, \quad \varphi'_{c+1} := \text{Committed}, \quad e'_{c+1} := 0, \quad \alpha' := \tau' := \kappa.$$

When the head has streamed its entire final snapshot, commitment is pure bookkeeping—no write occurs, because every row is already in both histories. Note the guard deliberately omits $\neg shut$: draining the head must remain possible during graceful shutdown.

3.5 Retirement

BEGINFLUSH. *Guard:* $\neg \text{flush}$. *Effect:* $\text{flush}' := \text{true}$. The flag is never reset.

Definition 3.3 (Retirement batch). For $b \in (c+1)..N$ with $\varphi_j = \text{Finalized}$ for all $j \in c+1..b$, the *logical* and *physical retirement rows* are

$$F_{\log}(b) = \text{tag}_{c+1}(f_{c+1}[e_{c+1}+1..|f_{c+1}|]) \cdot \bigodot_{j=c+2}^b \text{tag}_j(f_j),$$

$$F_{\text{phys}}^q(b) = \text{ntag}_{\omega}^q(c+1, f_{c+1}[e_{c+1}+1..|f_{c+1}|]) \cdot \bigodot_{j=c+2}^b \text{ntag}_{\omega}^q(j, f_j), \quad q \in \{\text{Retire}, \text{FailedWrite}\}.$$

The first (head) block contributes only its *unemitted* suffix—rows already streamed by **APPENDSTABLE** are never re-emitted.

RETIRESUCCESS(b). *Guard:* $\neg \text{rep} \wedge b \in (c+1)..N \wedge (\forall j \in c+1..b : \varphi_j = \text{Finalized}) \wedge \text{req}$. *Effect:*

$$\begin{aligned} L' &:= L \cdot F_{\log}(b), & P' &:= P \cdot F_{\text{phys}}^{\text{Retire}}(b), \\ c' &:= b, & \varphi'_j &:= \text{Committed} \text{ and } e'_j := 0 \text{ for } j \leq b, & \alpha' &:= \tau' := \kappa. \end{aligned}$$

Retirement advances the frontier across a *contiguous finalized prefix* and is permitted only under demand (req): pressure (row demand exceeding the viewport, or too many uncommitted blocks) or an explicit flush. Section 4.5 shows the physical effect is realized by a single streamed write.

RETIREFAILURE(b, k). *Guard:* as **RETIRESUCCESS**(b), plus $k \in 0..|F_{\text{phys}}^{\text{FailedWrite}}(b)|$. *Effect:*

$$P' := P \cdot F_{\text{phys}}^{\text{FailedWrite}}(b)[1..k], \quad \text{run}' := \text{false}, \quad \text{stop}' := \text{WriteFailure}.$$

L, c, φ, e are *unchanged*: a partial terminal write deposits an arbitrary prefix of immutable finalized/stable rows into native scrollback, tagged **FailedWrite**, and the host halts. Because the frontier did not advance, no retry can duplicate rows and no later block can overtake the failed batch—failure is fail-stop, not fail-retry (Theorem 4.16).

3.6 Resize and synchronous bottom-first replay

RESIZE($\omega^\dagger, h^\dagger, p, k$), $\omega^\dagger \in \mathbb{W}$, $h^\dagger \in 0..H$, $p \in \{\text{Preserve}, \text{Append}, \text{Rebuild}\}$, $k \in 0..|Q|$. *Guard:* $\neg \text{shut} \wedge \nu < R_{\max} \wedge (\omega^\dagger \neq \omega \vee h^\dagger \neq h)$. *Effect:* let

$$p^\dagger = \begin{cases} p & \omega^\dagger \neq \omega \\ \text{Preserve} & \text{otherwise,} \end{cases} \quad \text{begin} \iff p^\dagger \neq \text{Preserve} \wedge (c > 0 \vee \text{ph}),$$

and set

$$\begin{aligned} \omega' &:= \omega^\dagger, & h' &:= h^\dagger, & \nu' &:= \nu + 1, & \alpha' &:= \tau' := \kappa \text{ at } (\varphi, f, e, \omega', h'), \\ P' &:= \begin{cases} \langle \rangle & p^\dagger = \text{Rebuild} \\ P \cdot \text{ntag}_{\omega}^{\text{Resize}}(Q[1..k]) & \text{otherwise,} \end{cases} & \varepsilon' &:= \begin{cases} \varepsilon + 1 & p^\dagger = \text{Rebuild} \\ \varepsilon & \text{otherwise,} \end{cases} \\ (m'_r, \ell'_r, u'_r, p'_r) &:= \begin{cases} (p^\dagger, 1, c, \text{if ph then } e_{c+1} \text{ else } 0) & \text{begin} \\ (m_r, \ell_r, u_r, p_r) & \neg \text{begin} \wedge \text{rep} \\ (\text{None}, 0, 0, 0) & \text{otherwise,} \end{cases} \\ \text{prep}' &:= \text{false}, & \text{cut}' &:= 0. \end{aligned}$$

Height-only resizes are always **Preserve**. The parameter k captures *emulator-induced push*: when the host terminal itself scrolls up to k top viewport rows into its scrollbar during the resize (e.g. on height shrink), those rows are recorded in P tagged **Resize**—the logical ledger is untouched (Theorem 4.16, *resize logical-neutrality*). Under **Append**, committed history is re-rendered at the new width and appended; under **Rebuild**, the display epoch is reset and native history is reconstructed from scratch. A resize arriving *during* an unfinished replay preserves the replay window but invalidates any prepared frame.

PREPAREREPLAY. *Guard:* $\text{rep} \wedge \neg \text{prep}$. *Effect:* $\text{prep}' := \text{true}$, $\text{cut}' := \text{cut}^*$. Preparation samples the current viewport, computes the replay frame $\mathcal{R}$, and splits it at the required cut: the tail $\mathcal{R}[\text{cut}^*+1..]$ will be painted *bottom-first* into the blank region (no scroll, no history effect), and only the prefix $\mathcal{R}[1..\text{cut}^*]$ will scroll.

REPLAYSYNCHRONOUSUCCESS. *Guard:* $\text{rep} \wedge \text{prep}$. *Effect:*

$$P' := P \cdot \mathcal{R}[1..\text{cut}], \quad (m'_r, \ell'_r, u'_r, p'_r, \text{prep}', \text{cut}') := (\text{None}, 0, 0, 0, \text{false}, 0).$$

REPLAYSYNCHRONOUSFAILURE(k), $k \in 0..\text{cut}$. *Guard:* $\text{rep} \wedge \text{prep}$. *Effect:* $P' := P \cdot \mathcal{R}[1..k]$, $\text{run}' := \text{false}$, $\text{stop}' := \text{WriteFailure}$.

3.7 Shutdown

BEGINGRACEFULSHUTDOWN. *Guard:* $\neg \text{shut}$. *Effect:* for every i with $\varphi_i \neq \text{Absent}$,

$$\begin{aligned} \varphi'_i &:= \begin{cases} \text{Committed} & i \leq c \\ \text{Finalized} & \text{otherwise,} \end{cases} & f'_i &:= \begin{cases} f_i & i \leq c \vee \varphi_i = \text{Finalized} \\ w_i & \text{otherwise,} \end{cases} \\ \alpha' &:= \tau' := \kappa \text{ at } (\varphi', f', e, \omega, h), & \text{flush}' &:= \text{true}, & \text{shut}' &:= \text{true}. \end{aligned}$$

All queued and active work is frozen at its current speculation ($f := w$) and the pipeline is put under permanent flush; the remaining trajectory can only drain (stream the head, commit, replay) and exit.

GRACEFULEXIT(k), $k \in \{0, 1\}$. *Guard:* $\text{shut} \wedge \neg \text{rep} \wedge c = \gamma \wedge (k = 0 \vee h > 0)$. *Effect:* $\text{run}' := \text{false}$, $\text{stop}' := \text{Graceful}$, and if $k = 1$, $P' := P \cdot \text{ntag}_\omega^{\text{Exit}}(\langle Q[1] \rangle)$. Exit is permitted only after every created block has committed and any replay has drained; the optional single pushed row (tagged **Exit**) models the final scroll that blanks the viewport and restores the shell prompt.

DETACHEXIT(k), $k \in \{0, 1\}$. *Guard:* $\neg \text{shut} \wedge (k = 0 \vee h > 0)$. *Effect:* as **GRACEFULEXIT** with $\text{stop}' := \text{Detach}$. Detachment abandons uncommitted work without emitting it; the exactness equation (1) guarantees the ledger holds precisely the committed content at the instant of detachment.

3.8 Scheduler gate and fairness

Definition 3.4 (Next-state relation and specification). Let $\mathcal{T}$ be the set of all transitions above except the two replay writes. The next-state relation is the *gated* disjunction

$$\text{Next} \equiv \begin{cases} \text{REPLAYSYNCHRONOUSUCCESS} \vee \exists k : \text{REPLAYSYNCHRONOUSFAILURE}(k) & \text{prep} \\ \vee_{T \in \mathcal{T}} \exists \text{params} : T & \neg \text{prep}. \end{cases}$$

The specification is

$$\begin{aligned} \text{Spec} \equiv & \text{Init} \wedge \Box[\text{Next}]_{\text{vars}} \wedge \text{WF}(\text{RETIRESUCCESS}^\exists) \wedge \text{WF}(\text{PREPAREREPLAY}) \\ & \wedge \text{WF}(\text{REPLAYSYNCHRONOUSUCCESS}) \wedge \text{WF}(\text{APPENDSTABLE}) \\ & \wedge \text{WF}(\text{COMPLETEAPPENDONLY}), \end{aligned}$$

where $\text{RETIRESUCCESS}^\exists = \exists b : \text{RETIRESUCCESS}(b)$ and WF is weak fairness: an action that is eventually forever enabled is taken infinitely often.

The gate is the formal content of the word *synchronous*: once a replay frame is prepared, *no* other transition—no update, no allocation, no repaint-inducing step—may interleave before the single buffered write completes or fails. Section 4.6 shows this gate is load-bearing: the prepared-frame invariant $\text{cut} = \text{cut}^*$ would not be stable without it.

3.9 Conformance summary

Table 2 summarizes, per transition, the physical effects a conforming implementation *must* produce and those it *must not* produce. “Repaint only” means: the implementation may rewrite the h live viewport rows in place but must not cause any row to enter scrollbar and must not touch the ledger.

4 Formal Theorems and Proofs

4.1 Invariant catalogue

Definition 4.1 (State invariants). Define the following predicates on states.

- **(T)** *Typing*: every component lies in its range of Table 1.
- **(LS)** *Lifecycle shape*: $c \leq \gamma$; and for every $i \in \mathcal{B}$,

$$\varphi_i \in \begin{cases} \{\text{Committed}\} & i \leq c \\ \{\text{Queued, Active, Finalized}\} & c < i \leq \gamma \\ \{\text{Absent}\} & i > \gamma, \end{cases} \quad \mu_i \neq \perp \iff i \leq \gamma.$$

- **(SD)** *Snapshot discipline*: if $\varphi_i \in \{\text{Finalized, Committed}\}$ then $f_i \in \mathbb{S}$ and $f_i = w_i$; otherwise $f_i = \perp$.
- **(ED)** *Emission discipline*: for all i , $e_i \leq |w_i|$; $\mu_i \neq \text{AppendOnly} \Rightarrow e_i = 0$; and $e_i > 0 \Rightarrow i = c+1 \wedge \varphi_i \in \{\text{Active, Finalized}\}$.
- **(CAP)** *Capacity*: $\mathcal{A}(\alpha, \tau)$ at $(\varphi, f, e, \omega, h)$.
- **(ECH)** *Exact committed history*: equation (1), $L = C(c) \cdot A(c)$.
- **(RS)** *Replay shape*: if $m_r = \text{None}$ then $\ell_r = u_r = p_r = \text{cut} = 0$ and $\neg \text{prep}$; if $m_r \neq \text{None}$ then $\ell_r = 1$, $u_r \in 0..c$, $p_r \leq L_{\max}$, and: if prep then $\text{cut} = \text{cut}^*$ and $|\mathcal{R}[\text{cut}+1..|\mathcal{R}|]| \leq \text{room}$, else $\text{cut} = 0$.
- **(NSS)** *Native source safety*: for every native row (q, o, x, v) in P :

Transition	Required effect	Forbidden effect
CREATE, UPDATE, REQUESTALLOCATION, BEGINFLUSH, BEGINGRACEFULSHUTDOWN	repaint only (viewport may re-layout)	any scrollbar emission; any ledger append; frontier motion
ADMIT	repaint only; new 1-row slot appears at bottom	admission that overflows the viewport; slot for a non-FIFO block
APPLYALLOCATION	repaint at bridged height $B(\alpha_i, \tau_i)$	exceeding the joint reservation (2); shrinking $k > 2 \rightarrow 1$ without the 2-row frame
FINALIZEACTIVE, FINALIZEQUEUED	repaint only; all slots collapse to canonical 1-row previews	scrollback emission; any later mutation of f_i
APPENDSTABLE	scroll exactly $ \rho_\omega(\langle w_{c+1}[n] \rangle) $ rows carrying the next stable head row; ledger append of that row	streaming any block other than the head $c+1$; skipping or reordering rows; streaming without row pressure
COMPLETEAPPENDONLY	bookkeeping only (frontier +1)	re-emission of already-streamed rows
RETIRESUCCESS(b)	one streamed write $T = F_{\text{phys}}^{\text{Retire}}(b) \cdot Q'$ scrolling exactly $ F_{\text{phys}}^{\text{Retire}}(b) $ rows (Cor. 4.9)	retiring a non-contiguous or non-finalized range; re-emitting the emitted head prefix; retiring without req
RETIREFAILURE	halt; native holds a prefix of the FailedWrite-tagged batch	advancing c ; mutating the ledger; retrying the write
RESIZE (Preserve)	record emulator-pushed rows (tagged Resize); recanonicalize layout	replaying; mutating ledger, frontier, modes, wants, finals, or emitted counts
RESIZE (Append)	as Preserve, then schedule replay of $C(c) + \text{head prefix at new width}$	same
RESIZE (Rebuild)	clear display epoch ($P := \langle \rangle, \varepsilon+1$), schedule replay	same
PREPAREREPLAY + REPLAYSYNCHRONOUSSUCCESS	paint tail $\mathcal{R}[\text{cut}^*+1..]$ into blank rows bottom-first (no scroll), then <i>one</i> synchronous buffered write scrolling exactly cut^* rows	interleaving any repaint between preparation and the write; multi-frame incremental replay; scrolling more or fewer than cut^* rows
REPLAYSYNCHRONOUSFAILURE	halt; native holds a prefix of $\mathcal{R}[1.. \text{cut}]$	continuing after a partial write
GRACEFULEXIT, DETACHEXIT	optionally scroll the single top viewport row; restore terminal state	emitting speculative content of uncommitted blocks; exiting gracefully with $c < \gamma$ or during replay

Table 2: Conformance obligations per transition.

- $q = \text{Retire} \Rightarrow o \in [c] \wedge \varphi_o = \text{Committed}$;
- $q \in \{\text{Append}, \text{Replay}\} \Rightarrow o \in \mathcal{B} \wedge (o \in [c] \vee (o = c+1 \wedge \mu_o = \text{AppendOnly}))$;
- $q = \text{FailedWrite} \Rightarrow \text{stop} = \text{WriteFailure}$;
- $q = \text{Exit} \Rightarrow \neg \text{run}$.

Let $J \equiv T \wedge \text{LS} \wedge \text{SD} \wedge \text{ED} \wedge \text{CAP} \wedge \text{ECH} \wedge \text{RS} \wedge \text{NSS}$.

Two further invariants—*no premature history* (NPH) and *screen capacity* (SC)—are checked independently by TLC but are derivable from J ; we prove them as Corollaries 4.14 and 4.15.

4.2 Exact capacity and overflow bounds

Theorem 4.2 (Exact capacity and overflow forcing). *Let a state satisfy CAP and SD. Then:*

- (i) **Exactness.** *Q is well defined, $|Q| = h$, block i owns exactly α_i cells of Q , exactly σ cells equal $\omega_{\boxplus}$, and exactly $h - \sum_i \alpha_i - \sigma \geq 0$ cells equal β .*
- (ii) **Zero-height degradation.** *If $h = 0$ then $\alpha_i = \tau_i = 0$ for all i , $\sigma = 0$, and $Q = \langle \rangle$; every transition remains well defined.*
- (iii) **Overflow forcing.** *If ovf and $h > 0$, then the visible blocks are exactly the $h - 1$ presented blocks of largest index, and necessarily $\alpha_i = \tau_i = 1$ for each of them, $\sigma = 1$, and Q contains no blank cell: the layout is forced to $Q = \langle \omega_{\boxplus} \rangle \cdot \text{cell}(i_{h-1}) \cdots \text{cell}(i_1)$ where $i_1 > \cdots > i_{h-1}$ enumerate the visible blocks in decreasing order.*

Proof. (i). Let $A_\Sigma = \sum_i \alpha_i$. Since $\alpha_i \leq \max(\alpha_i, \tau_i)$ pointwise, (2) gives $A_\Sigma \leq \text{Res}(\alpha, \tau) \leq h - \sigma$, so the blank exponent $h - A_\Sigma - \sigma$ in (3) is a natural number. The three factors of (3) contribute $h - A_\Sigma - \sigma$, σ , and $\sum_i \alpha_i$ cells respectively, totalling h . Ownership: the slot factor $\text{cell}(i)^{\alpha_i}$ contributes exactly α_i cells with owner i , and blank/overflow cells carry owner $0 \notin \mathcal{B}$. Well-definedness of $\text{cell}(i)$ for $\alpha_i > 0$: by CAP clause 2, $\alpha_i > 0$ implies $\text{vis}(i)$, hence $\text{pres}(i)$, hence $\varphi_i \in \{\text{Active}, \text{Finalized}\}$; if $\varphi_i = \text{Active}$ then $\text{pv}(i) = U_i(w_i)$ with $w_i \in \mathbb{S}$; if $\varphi_i = \text{Finalized}$ then by SD $f_i \in \mathbb{S}$, so $\text{pv}(i) = U_i(f_i)$ is a genuine row sequence. The placeholder branch covers an empty rendering.

(ii). Let $h = 0$. If $\pi = 0$ no block is presented, so no block is visible and CAP clause 2 zeroes all α, τ . If $\pi > 0$ then $\pi > 0 = h$, i.e. ovf; visibility then requires $h > 0$, which fails, so again no block is visible and all allocations are zero; moreover $\sigma = 0$ by its $h > 0$ guard. Hence $Q = \beta^0 \cdot \langle \rangle \cdot \langle \rangle = \langle \rangle$. Every transition guard and effect refers to Q , room, and allocation predicates that are all well defined at $h = 0$ (in particular GRACEFULEXIT/DETACHEXIT require $k = 0$ when $h = 0$, and room = 0 makes any replay scroll its entire frame, $\text{cut}^* = |\mathcal{R}|$).

(iii). Assume ovf and $h > 0$, so $\pi > h \geq 1$ and $\sigma = 1$. List the presented blocks in decreasing index order $i_1 > i_2 > \cdots > i_\pi$; then $\text{newer}(i_k) = k - 1$, so $\text{vis}(i_k) \iff k - 1 < h - 1 \iff k \leq h - 1$. Since $\pi \geq h + 1 > h - 1$, exactly $h - 1$ blocks are visible, namely $i_1, \dots, i_{h-1}$. By CAP clause 1 each visible block has $\max(\alpha_i, \tau_i) \geq \alpha_i \geq 1$; summing, $\text{Res}(\alpha, \tau) \geq h - 1$, while (2) gives $\text{Res}(\alpha, \tau) \leq h - \sigma = h - 1$. Hence every visible block has $\max(\alpha_i, \tau_i) = 1$, so $\alpha_i = 1$ and $\tau_i \leq 1$; for active visible blocks CAP requires $\tau_i \geq 1$, and for finalized visible blocks $\tau_i = \alpha_i = 1$. Thus $\alpha_i = \tau_i = 1$ on the visible set and 0 elsewhere; $A_\Sigma = h - 1$; the blank exponent is $h - (h - 1) - 1 = 0$; and (3) degenerates to the stated forced layout, the slots appearing in ascending block order, i.e. the reversed enumeration. $\square$

Corollary 4.3 (Canonical admissibility). *For every ambient tuple $(\varphi, f, e, \omega, h)$ (with SD for well-definedness), $\mathcal{A}(\kappa, \kappa)$ holds.*

Proof. Clauses 1–2 of Definition 2.10 hold by construction of κ , provided $1 \leq H$ whenever some block is visible; visibility of any block implies $h \geq 1$ (if $\neg\text{ovf}$ then $1 \leq \pi \leq h$; if ovf then visibility itself demands $h > 0$), and $h \leq H$. For (2): $\text{Res}(\kappa, \kappa) = |\{i : \text{vis}(i)\}|$. If $h = 0$ this is $0 = h - \sigma$ by Theorem 4.2(ii). If ovf and $h > 0$ it is $h - 1 = h - \sigma$ by (iii). If $\neg\text{ovf}$ every presented block is visible, so the count is $\pi \leq h = h - \sigma$. $\square$

Corollary 4.3 is used ubiquitously below: every transition that recanonicalizes $(\alpha' := \tau' := \kappa)$ re-establishes CAP unconditionally, at whatever successor tuple it evaluates κ .

4.3 Reservation dominance and bridge termination

Lemma 4.4 (Reservation dominance; bridge termination). *Let $a \neq t$ with $a, t \in 0..H$ and let B be the bridge function of Definition 3.2. Then:*

- (i) Dominance: $\max(B(a, t), t) \leq \max(a, t)$; consequently an APPLYALLOCATION step never increases any summand of $\text{Res}(\alpha, \tau)$, and the reservation invariant (2) is preserved verbatim (the ambient tuple, hence σ and h , is unchanged by the step).
- (ii) Positivity: if $t \geq 1$ then $B(a, t) \geq 1$; hence an active visible block never transits through a 0-height frame.
- (iii) Termination: for fixed t , the iteration $a_0 = a$, $a_{n+1} = B(a_n, t)$ reaches t in at most two steps, and each step strictly changes the allocation. Precisely: if $a < t$ or $t \geq 2$ or $a \leq 2$, one step suffices; if $a > 2$ and $t = 1$, the trajectory is $a \rightarrow 2 \rightarrow 1$.

Proof. (i). Case $a < t$ (growth): $B = t$ and $\max(t, t) = t = \max(a, t)$. Case $a > t$: if $a > 2 \wedge t = 1$ then $B = 2$ and $\max(2, 1) = 2 < a = \max(a, t)$ since $a \geq 3$; otherwise $B = t$ and $\max(t, t) = t < a = \max(a, t)$. In all cases the per-block reservation summand does not increase; all other summands are untouched, and σ, h depend only on $(\varphi, f, e, \omega, h)$, which APPLYALLOCATION leaves unchanged.

(ii). If $a < t$ then $B = t \geq 1$; otherwise $B \in \{t, 2\}$ and both are ≥ 1 when $t \geq 1$.

(iii). If $a < t$: $B(a, t) = t$, one step. If $a > t$ and $\neg(a > 2 \wedge t = 1)$: $B(a, t) = t$, one step. If $a > 2$ and $t = 1$: $B(a, 1) = 2 \neq a$ (as $a \geq 3$), then $B(2, 1)$ falls in the “otherwise” branch (since $2 \not> 2$) giving 1. Each step changes α_i : the guard $\alpha_i \neq \tau_i$ excludes stuttering, and $B(a, t) \neq a$ in every case above. $\square$

Remark 4.5. Dominance is exactly why REQUESTALLOCATION charges $\max(\alpha_i, \tau_i^\dagger)$ at request time: once a target is admitted, every frame of the bridge animation—including the stabilizing 2-row frame of a deep shrink—is guaranteed to fit without re-negotiation, regardless of how apply steps of distinct blocks interleave.

4.4 Stable-head emission and ledger monotonicity

Lemma 4.6 (Stable-head emission). *Assume J. Then:*

- (i) Prefix immutability. *For every transition and every $i \in \mathcal{B}$, if $e_i > 0$ then $w'_i[1..e_i] = w_i[1..e_i]$ whenever $e'_i \geq e_i$; and the only transitions with $e'_i < e_i$ (COMPLETEAPPENDONLY, RETIRESUCCESS) commit block i in the same step.*

- (ii) Exactness preservation by streaming. `APPENDSTABLE` preserves ECH: $L' = C(c) \cdot A'(c)$.
- (iii) No re-emission. Across any behavior, each position of each block's final snapshot contributes at most one ledger row and at most one batch of `Append/Retire` native rows: rows emitted by `APPENDSTABLE` are excluded from the block's retirement batch (Definition 3.3 slices from $e_{c+1}+1$), and after commitment $e_i = 0$ with block i never again the head.
- (iv) Ledger monotonicity. Every transition satisfies $L \sqsubseteq L'$.

Proof. (i). By ED, $e_i > 0$ implies $\mu_i = \text{AppendOnly}$, $i = c+1$, and $\varphi_i \in \{\text{Active}, \text{Finalized}\}$. The transitions that modify w_i are `UPDATE` and the two finalizations, each guarded (for $\mu_i = \text{AppendOnly}$) by $w_i \sqsubseteq s$ with $w'_i := s$; a prefix extension preserves every position $\leq e_i \leq |w_i|$. No other transition writes w . The transitions that decrease e_i are exactly `COMPLETEAPPENDONLY` and `RETIRESUCCESS`, both of which set $\varphi'_i = \text{Committed}$ for the affected block(s).

(ii). The guard gives $e_{c+1} < |w_{c+1}|$ and, writing $n = e_{c+1} + 1$,

$$L' = L \cdot \langle (c+1, w_{c+1}[n]) \rangle = C(c) \cdot A(c) \cdot \langle (c+1, w_{c+1}[n]) \rangle = C(c) \cdot \text{tag}_{c+1}(w_{c+1}[1..n]),$$

using ECH for the middle equality and, for the last, that $A(c) = \text{tag}_{c+1}(w_{c+1}[1..e_{c+1}])$ when $e_{c+1} > 0$ and $A(c) = \langle \rangle$ when $e_{c+1} = 0$ (in either case the concatenation is the tagged prefix of length n). In the successor, $e'_{c+1} = n > 0$, and $\mu, \varphi_{c+1}, c, \gamma$ are unchanged, so ph' holds and $A'(c) = \text{tag}_{c+1}(w_{c+1}[1..n])$. Hence $L' = C(c) \cdot A'(c)$.

(iii). A row of block i enters L either via `APPENDSTABLE` at position e_i+1 (incrementing e_i) or via `RETIRESUCCESS`, whose logical batch for the head starts at position $e_{c+1}+1$ and covers later blocks in full—blocks $j > c+1$ have $e_j = 0$ by ED. By ECH the ledger is at all times *equal* to $C(c) \cdot A(c)$, in which position p of block i occurs at most once; monotonicity of c and of e_{c+1} between commits (part (i)) makes the occurrence permanent. The same slicing governs the physical batches.

(iv). Only `APPENDSTABLE` and `RETIRESUCCESS` write L , both by right-concatenation. $\square$

4.5 Generic terminal streaming realization

We now connect the abstract effect “append F to P and repaint the viewport as Q' ” to the concrete mechanics of a scrolling terminal.

Definition 4.7 (Abstract scrolling terminal). A *terminal* of height $h \geq 0$ is a pair (S, V) with S a row sequence (scrollback) and V a row sequence of length h (viewport). The *print* operation writes a row sequence T starting at viewport row $r_0 \in [h] \cup \{h+1\}$ (when $h = 0$, $r_0 = 1$ and every written row scrolls): rows are written top-to-bottom at consecutive positions; writing at a position $r \leq h$ replaces $V[r]$; when the cursor would advance past row h , the terminal *scrolls*: $S := S \cdot \langle V[1] \rangle$, $V := V[2..h] \cdot \langle \square \rangle$, and the cursor stays at row h .

Lemma 4.8 (Streaming realization). Let (S, V) be a terminal of height h , let F be any row sequence and Q any row sequence with $|Q| = h$, and let $T = F \cdot Q$, $k = |F|$. Printing T from row $r_0 = 1$ when $k \geq 1$, or overwriting rows $1..h$ when $k = 0$, yields the terminal $(S \cdot F, Q)$: the scrollback is extended by exactly F , in order, and the final viewport equals Q . No row of the pre-state viewport V enters S .

Proof. For $k = 0$ the print rewrites $V[1..h] := Q$ with no cursor overflow; S is unchanged and $S \cdot F = S$.

For $k \geq 1$ we prove by induction on $j \in 0..k$ the invariant: after writing the first $h + j$ rows of T ,

$$S_j = S \cdot T[1..j], \quad V_j = T[j+1..j+h].$$

Base $j = 0$: the first h writes fill rows $1..h$ with $T[1..h]$ without overflow, so $S_0 = S$ and $V_0 = T[1..h]$.

Step: to write row $h+j+1$ of T the cursor advances past row h ; the terminal scrolls, moving $V_j[1] = T[j+1]$ into scrollbar and shifting, then the write places $T[h+j+1]$ at row h :

$$\begin{aligned} S_{j+1} &= S_j \cdot \langle T[j+1] \rangle = S \cdot T[1..j+1], \\ V_{j+1} &= T[j+2..j+h] \cdot \langle T[h+j+1] \rangle = T[(j+1)+1..(j+1)+h]. \end{aligned}$$

At $j = k$: $S_k = S \cdot T[1..k] = S \cdot F$ and $V_k = T[k+1..k+h] = Q$. Rows scrolled out are exactly $T[1..k] = F$; every row of the old viewport is overwritten in the base phase before any scroll, so none reaches S . When $h = 0$, $T = F$, every write scrolls, and the claim is the base-free instance of the same induction. $\square$

Corollary 4.9 (Retirement realization). *RETIRESUCCESS(b) is realized on a terminal displaying Q by a single streamed write of $T = F_{\text{phys}}^{\text{Retire}}(b) \cdot Q'$ (rows taken as the row components of the tagged sequences), scrolling exactly $|F_{\text{phys}}^{\text{Retire}}(b)|$ times; likewise APPENDSTABLE with $F = \rho_\omega(\langle w_{c+1}[n] \rangle)$ (1 or 2 physical rows), and GRACEFULEXIT/DETACHEXIT with $k = 1$ and $F = \langle Q[1] \rangle$ followed by a blanked viewport. In each case the scrollbar extension equals, row for row and in order, the native rows appended by the transition, and no speculative viewport content leaks into scrollbar.*

Proof. Instantiate Lemma 4.8 with $Q = Q'$, which has length h by Theorem 4.2(i) applied in the successor state (CAP holds there by Corollary 4.3). The transition's native append is by definition the tagged image of F . The final claim is the lemma's “no row of V enters S ”. $\square$

Remark 4.10 (Fail-stop realization). *RETIREFAILURE(b, k) is the same write interrupted after k physical rows: by Lemma 4.8's induction, after $\min(k, \dots)$ writes the scrollbar extension is exactly a prefix of $F_{\text{phys}}^{\text{FailedWrite}}(b)$ —never a row of Q' , never out of order. Because the abstract transition does not advance c or touch L , and $\text{run}' = \text{false}$ disables every subsequent transition (Theorem 4.16(vii)), the physical scrollbar ends with a clean, attributable, immutable prefix of finalized content.*

4.6 Bottom-first synchronous replay

Lemma 4.11 (Bottom-first synchronous replay invariant). *Assume J and rep . Then:*

- (i) Split correctness. *After PREPAREREPLAY, $\text{cut} = \text{cut}^*$ and the prepared tail satisfies $|\mathcal{R}[\text{cut}+1..|\mathcal{R}|]| = \min(|\mathcal{R}|, \text{room}) \leq \text{room}$.*
- (ii) History-neutral tail paint. *Painting the prepared tail into the blank region of Q (bottom-anchored, by absolute cursor addressing) writes only rows at viewport positions occupied by β cells and causes no scroll; hence it extends neither P nor L .*
- (iii) Single-write scroll exactness. *The subsequent synchronous buffered write is the streaming realization (Lemma 4.8) of $F = \mathcal{R}[1..\text{cut}]$ with final viewport the replay-completed frame; it scrolls exactly cut rows, and the native extension is exactly $\mathcal{R}[1..\text{cut}]$ —as specified by REPLAYSYNCHRONOUSUCCESS.*

- (iv) Gate necessity and atomicity. *Between PREPAREREPLAY and the write, the scheduler gate of Definition 3.4 admits no other transition; this is necessary: absent the gate, an interleaved ADMIT, UPDATE, FINALIZEQUEUED, or allocation step can change room or $|\mathcal{R}|$ and falsify $cut = cut^*$, so the invariant RS is stable only under the gate. Consequently at most one frame is ever presented per replay: no multi-frame flicker and no torn intermediate state is observable.*
- (v) Failure safety. *REPLAYSYNCHRONOUSFAILURE(k) leaves scrollbar extended by the prefix $\mathcal{R}[1..k]$, $k \leq cut$, of re-rendered committed rows (and stable head prefix); the ledger and frontier are untouched and the host halts.*

Proof. (i). PREPAREREPLAY sets $cut' := cut^*$ by definition. Then

$$|\mathcal{R}| - cut^* = |\mathcal{R}| - \max(0, |\mathcal{R}| - \text{room}) = \min(|\mathcal{R}|, \text{room}) \leq \text{room}.$$

(ii). By (i) the tail occupies at most room rows and room counts exactly the β positions of Q (Theorem 4.2(i) makes this count well defined and the blank block contiguous at the top of Q , positions $1..h - A_\Sigma - \sigma$). Writing $\leq \text{room}$ rows at absolute positions within $[1..h]$ never advances the cursor past row h , so by Definition 4.7 no scroll occurs and S is unchanged. The transition PREPAREREPLAY indeed changes neither P nor L .

(iii). Instantiate Lemma 4.8 with $F = \mathcal{R}[1..cut]$ and Q the post-replay physical frame (the h -row image containing the painted tail and the live slots). The scrollbar extension is exactly F , matching the abstract effect $P' := P \cdot \mathcal{R}[1..cut]$ of REPLAYSYNCHRONOUSSUCCESS; L , c , w , f , e are unchanged, so replay is logically neutral.

(iv). By Definition 3.4, when $prep$ holds the next-state relation is the disjunction of the two replay writes only. For necessity, observe that cut^* is a function of $|\mathcal{R}|$ and room; room is a function of α and σ , both of which are changed by ADMIT (new 1-row slot), APPLYALLOCATION, FINALIZEQUEUED (possible new presented block, possibly overflow), and UPDATE on the head can change $|\mathcal{R}|$ via w_{ur+1} —so RS's clause $prep \Rightarrow cut = cut^*$ would not be preserved by those steps. Physically, the gate is the statement that the implementation assembles the remainder $F \cdot Q$ into one buffer and issues it as one synchronous write: no repaint can be scheduled between preparation and completion, so the display transitions from the pre-replay frame directly to the completed frame.

(v). Immediate from the transition's definition and Lemma 4.8's prefix property; NSS's Replay clause (proved in Theorem 4.13) attributes every such row to committed content or the append-only head. $\square$

Remark 4.12 (Bottom-first order). The order—tail first, scroll-write second—is what makes the *visible* viewport correct at every instant: the tail paint (ii) fills blank rows that carry no information, and the single write (iii) atomically supplies the scrolled prefix and the final frame. Painting top-first (prefix before tail) would either require cut speculative scrolls before the tail exists on screen (tearing: transient frames showing history without its continuation) or force multi-frame incremental replay (flicker). The model's gate plus split renders both failure modes unrepresentable.

4.7 Global inductive safety

Theorem 4.13 (Global inductive safety). *J (Definition 4.1) is an inductive invariant of Spec: the initial state satisfies J , and every transition of Section 3, fired in any state satisfying J , yields a state satisfying J . Hence $\text{Spec} \models \square J$.*

Proof. Initiality. In the state of Definition 2.6: T holds by inspection; LS with $c = \gamma = 0$; SD with all phases **Absent** and $f \equiv \perp$; ED with $e \equiv 0$; CAP because no block is presented, so $\kappa \equiv 0 = \alpha = \tau$ and $\text{Res} + \sigma = 0 \leq H = h$ (Corollary 4.3); ECH because $L = \langle \rangle = C(0) \cdot A(0)$ (ph fails at $c = \gamma$); RS by the **None** clause; NSS vacuously ($P = \langle \rangle$).

Preservation. Fix a state satisfying J and a transition. We argue each conjunct; components not written by the transition preserve the corresponding clauses trivially, so we treat only the writers.

T. All effects assign values within the declared ranges: phases/modes from their alphabets; $e'_{c+1} = e_{c+1} + 1 \leq |w_{c+1}| \leq L_{\max}$ in **APPENDSTABLE** (guard); α', τ' from guards or $\kappa \in \{0, 1\}^B$; $c' = b \leq N$; $\nu' \leq R_{\max}$ (guard) and $\varepsilon' \leq \nu' \leq R_{\max}$ (an epoch increment occurs only within a resize); $\text{cut}' = \text{cut}^* \leq |\mathcal{R}| \leq 2(cL_{\max} + L_{\max}) \leq 2NL_{\max} = K_{\max}$ using Proposition 2.3(i); ledger/native rows are constructed by tag/ntag over their row types.

LS. **CREATE** sets $\varphi_{\gamma+1}$ from **Absent** to **Queued** with a mode, extending the created range contiguously by one; the bands shift accordingly. **ADMIT**, **UPDATE**, the finalizations, and **BEGINGRACEFULSHUTDOWN** keep every touched block inside the band prescribed for its index: indices $\leq c$ are untouched or (shutdown) set to **Committed**, indices in $(c.. \gamma]$ move within $\{\text{Queued}, \text{Active}, \text{Finalized}\}$. **COMPLETEAPPENDONLY** and **RETIRESUCCESS**(b) set $\varphi'_j = \text{Committed}$ exactly for $j \leq c' \in \{c+1, b\}$ and leave later blocks in place; $c' \leq \gamma$ because the guard requires those blocks finalized, hence created. No transition unsets a mode or reverts a phase.

SD. Writers of f : the finalizations set $f'_i = s = w'_i$ with $\varphi'_i = \text{Finalized}$ (and by LS the block was not finalized/committed before, so no frozen final is overwritten); **BEGINGRACEFULSHUTDOWN** sets $f'_i = w_i$ exactly for blocks moving **Queued/Active** $\rightarrow$ **Finalized** (for which w is unchanged, giving $f'_i = w'_i$) and preserves f_i for already finalized/committed blocks, where SD gave $f_i = w_i$ and w is unchanged. Writers of w with a frozen final: none—**UPDATE** requires $\varphi_i \in \{\text{Queued}, \text{Active}\}$ (so $f_i = \perp$), and the finalizations set both together. Commitment (**COMPLETEAPPENDONLY**, **RETIRESUCCESS**) changes phases **Finalized** $\rightarrow$ **Committed** with f, w untouched.

ED. Writers of e : **APPENDSTABLE** increments e_{c+1} within $|w_{c+1}|$ (guard), for a block with $\mu = \text{AppendOnly}$, $\varphi \in \{\text{Active}, \text{Finalized}\}$, index $c+1$, and leaves c fixed; **COMPLETEAPPENDONLY** and **RETIRESUCCESS** zero e_j for all $j \leq c'$ while advancing c —by ED any i with $e_i > 0$ satisfied $i = c+1 \leq c'$, so no positive e survives at an index $\neq c'+1$; indices $> c'$ retain $e = 0$. Writers of w : for $\mu_i = \text{AppendOnly}$ the prefix guards give $|w'_i| \geq |w_i| \geq e_i$; for $\mu_i = \text{Mutable}$, ED gives $e_i = 0$. Writers of φ at an index with $e_i > 0$: only **FINALIZEACTIVE** (**Active** $\rightarrow$ **Finalized**, allowed) and **BEGINGRACEFULSHUTDOWN** (by ED the block with $e_i > 0$ has $\varphi_i \in \{\text{Active}, \text{Finalized}\}$ and $i = c+1 > c$, so it moves to **Finalized**, allowed).

CAP. **ADMIT**, **REQUESTALLOCATION**, and **APPLYALLOCATION** carry $\mathcal{A}$ at the successor configuration explicitly in their guards (for **APPLYALLOCATION** the guard is moreover always satisfiable along a bridge by Lemma 4.4(i)). **UPDATE** changes only w , on which $\mathcal{A}$ does not depend (visibility uses φ, f, e, ω, h only). **BEGINFLUSH**, **PREPAREREPLAY**, the replay writes, **RETIREFAILURE**, and the exits change none of $\alpha, \tau, \varphi, f, e, \omega, h$. Every remaining writer (**FINALIZEACTIVE**, **FINALIZEQUEUED**, **APPENDSTABLE**, **COMPLETEAPPENDONLY**, **RETIRESUCCESS**, **RESIZE**, **BEGINGRACEFULSHUTDOWN**) sets $\alpha' = \tau' = \kappa$ evaluated at its successor tuple, and Corollary 4.3 yields $\mathcal{A}$ there.

ECH. Writers of any of $L, c, f, w, e, \varphi, \mu$:

- **CREATE:** $\gamma' = \gamma + 1$. If $c < \gamma$, the head block $c+1$ is untouched, so ph and $A(c)$ are unchanged. If $c = \gamma$, then ph was false; afterwards $e_{c+1} = 0$ (ED), so ph' is false. Either way $L' = L = C(c) \cdot A(c)$.

- **ADMIT, APPLYALLOCATION, REQUESTALLOCATION,**
BEGINFLUSH, RESIZE, PREPAREREPLAY, the replay writes, failures, exits: none of L, c, f, w, e change, and ph depends only on $(c, \gamma, \mu_{c+1}, \varphi_{c+1}, e_{c+1})$; **ADMIT** can change φ_{c+1} only **Queued** $\rightarrow$ **Active**, which affects ph only when $e_{c+1} > 0$, impossible for a queued block by **ED**.
- **UPDATE(i, s):** only w_i changes. If $i \neq c+1$ or ph fails with $e_{c+1} = 0$, $A(c)$ is unaffected. If $i = c+1$ and $e_{c+1} > 0$, **ED** forces $\mu_i = \text{AppendOnly}$, the guard gives $w_i \sqsubseteq s$, hence $s[1..e_i] = w_i[1..e_i]$ and $A'(c) = A(c)$.
- **Finalizations:** f_i is set for $i > c$, so $C(c)$ is unchanged; w_i extends by prefix in the append-only case (as above) and $e_i = 0$ in the mutable case; $\varphi'_i = \text{Finalized}$ keeps ph 's phase disjunct satisfied when $i = c+1$.
- **APPENDSTABLE:** Lemma 4.6(ii).
- **COMPLETEAPPENDONLY:** the guard gives $e_{c+1} = |f_{c+1}|$ and **SD** gives $f_{c+1} = w_{c+1}$, so $A(c) = \text{tag}_{c+1}(w_{c+1}[1..e_{c+1}]) = \text{tag}_{c+1}(f_{c+1})$ (also when $|f_{c+1}| = 0$, both sides empty). Hence $L = C(c) \cdot \text{tag}_{c+1}(f_{c+1}) = C(c+1) = C(c')$. In the successor, every $e'_j = 0$ for $j \leq c'$ and **ED** excludes $e'_{c+1} > 0$ (only index $c+1$ could be positive), so ph' fails and $A'(c') = \langle \rangle$.
- **RETIRESUCCESS(b):** all blocks $c+1..b$ are finalized, so **SD** gives $f_j = w_j$ on the range. If $e_{c+1} > 0$ then $A(c) \cdot \text{tag}_{c+1}(f_{c+1}[e_{c+1}+1..]) = \text{tag}_{c+1}(f_{c+1})$; if $e_{c+1} = 0$ then $A(c) = \langle \rangle$ and the head batch is all of $\text{tag}_{c+1}(f_{c+1})$. In both cases

$$L' = C(c) \cdot A(c) \cdot F_{\log}(b) = C(c) \cdot \bigodot_{j=c+1}^b \text{tag}_j(f_j) = C(b) = C(c').$$

As above, all e'_j vanish for $j \leq b$ and $e_{b+1} = 0$ by **ED**, so $A'(c') = \langle \rangle$.

- **BEGINGRACEFULSHUTDOWN:** L, c, w, e unchanged; f_j for $j \leq c$ unchanged, so $C(c)$ unchanged; if ph held, the head's phase moves **Active** $\mapsto$ **Finalized** or stays **Finalized**, so ph' holds and A is unchanged (w, e fixed); if ph failed, it fails after (a positive e_{c+1} cannot appear).

RS. RESIZE: in the *begin* case it establishes the rep clause directly ($\ell'_r = 1, u'_r = c \in 0..c, p'_r \leq L_{\max}$ by **T**, prep' false, $\text{cut}' = 0$); continuing an existing replay it preserves the window (still $u_r \leq c$ since c is unchanged) and resets prep, cut to the unprepared shape; otherwise it writes the **None** shape. **PREPAREREPLAY** establishes the prepared clause: $\text{cut}' = \text{cut}^*$ and the tail bound by Lemma 4.11(i). *Stability of the prepared clause:* by the scheduler gate (Definition 3.4) the only transitions from a prepared state are the two replay writes; success restores the **None** shape and failure freezes the system, so no step can invalidate $\text{cut} = \text{cut}^*$ (Lemma 4.11(iv)). While unprepared and replaying, transitions other than **RESIZE/PREPAREREPLAY** leave the replay components untouched, and all c -advancing transitions (**COMPLETEAPPENDONLY, RETIRESUCCESS**) are disabled by their $\neg \text{rep}$ guards, preserving $u_r \leq c$.

NSS. New native rows, by transition: **APPENDSTABLE** appends **Append** rows with owner $c+1$ and $\mu_{c+1} = \text{AppendOnly}$ (guard); **RETIRESUCCESS(b)** appends **Retire** rows with owners in $c+1..b \subseteq [c']$, all committed in the successor; **RESIZE** appends **Resize** rows (no **NSS** obligation) or clears P (vacuous); the replay writes append **Replay** rows whose owners lie in $[u_r] \cup \{u_r+1\}$ with $u_r \leq c$ and, when $p_r > 0$, owner $u_r+1 = c+1$ with $\mu_{c+1} = \text{AppendOnly}$ —the latter because

$p_r > 0$ was set at a resize at which ph held, μ is immutable after creation, and c cannot advance while rep holds, so the identity $u_r = c$ persists from that resize; RETIREFAILURE and REPLAYSYNCHRONOUSFAILURE append FailedWrite- and Replay-tagged rows respectively while setting $stop' = \text{WriteFailure}$ and $run' = \text{false}$; the exits append Exit rows while setting $run' = \text{false}$. Old rows: the clauses for Retire, Append, Replay mention c, φ, μ , all of which evolve monotonically in the required direction— c never decreases, committed blocks stay committed, modes are immutable, and an owner $o = c+1$ of an Append/Replay row moves into $[c']$ whenever the frontier passes it. The FailedWrite/Exit clauses are preserved because $stop$ and run change only in halting transitions, after which (Theorem 4.16(vii)) the state is frozen. $\square$

Corollary 4.14 (No premature native or ledger rows). *In every reachable state, every ledger row is owned either by a committed block $o \in [c]$ (with $\varphi_o = \text{Committed}$) or, when ph holds, by the head $c+1$; and every Append/Retire/Replay native row is owned by a committed block or by the append-only head. In particular no row of a queued, merely-active mutable, or unfinalized non-head block ever reaches either history.*

Proof. By ECH, $L = C(c) \cdot A(c)$: rows of $C(c)$ have owners $j \in [c]$, which LS makes committed; rows of $A(c)$ exist only under ph and carry owner $c+1$. The native claim is NSS. Both hold in all reachable states by Theorem 4.13. $\square$

Corollary 4.15 (Screen capacity). *In every reachable state, Q is an h -row cell sequence in which block i owns exactly α_i rows, exactly σ rows carry the overflow marker, and exactly $h - \sum_i \alpha_i - \sigma$ rows are blank.*

Proof. Theorem 4.2(i) applied under $\square J$. $\square$

Theorem 4.16 (Action safety). *Every step of Spec (including stuttering) satisfies:*

- (i) Ledger monotonicity: $L \sqsubseteq L'$.
- (ii) Epoch-scoped native monotonicity: either $\varepsilon' = \varepsilon$ and $P \sqsubseteq P'$, or $\varepsilon' = \varepsilon + 1$ and $P' = \langle \rangle$.
- (iii) Final immutability: if $\varphi_i \in \{\text{Finalized}, \text{Committed}\}$ then $f'_i = f_i$.
- (iv) Append-only monotonicity: if $\mu_i = \text{AppendOnly}$ and $\varphi_i \in \{\text{Queued}, \text{Active}\}$ then $w_i \sqsubseteq w'_i$.
- (v) Resize logical-neutrality: if $\omega' \neq \omega$ or $h' \neq h$ then $L' = L$, $c' = c$, $\mu' = \mu$, $w' = w$, $f' = f$, $e' = e$.
- (vi) Fail-stop: $\square(\text{stop} = \text{WriteFailure} \Rightarrow \neg \text{run})$.
- (vii) Stopped quiescence: if $\neg \text{run}$ then the step is a stutter (all components unchanged).

Proof. (i) Lemma 4.6(iv); stutters are trivial. (ii) The writers of P are APPENDSTABLE, RETIRESUCCESS, RETIREFAILURE, RESIZE, the replay writes, and the exits; all append except RESIZE with $p^\dagger = \text{Rebuild}$, which simultaneously sets $P' = \langle \rangle$ and $\varepsilon' = \varepsilon + 1$; no other transition writes ε . (iii) The finalizations write f_i only for $\varphi_i \in \{\text{Active}, \text{Queued}\}$; BEGINGRACEFULSHUTDOWN preserves f_i whenever $i \leq c$ or $\varphi_i = \text{Finalized}$, which by LS covers all finalized and committed blocks. (iv) The writers of w_i are UPDATE and the finalizations; for $\mu_i = \text{AppendOnly}$ each is guarded by $w_i \sqsubseteq s$. (v) Only RESIZE writes ω or h , and its effect leaves the six listed components untouched. (vi) $stop$ is set to WriteFailure only by RETIREFAILURE and REPLAYSYNCHRONOUSFAILURE, both of which set $run' = \text{false}$ in the same step; by (vii) no later

step changes either component; initially $stop = \text{Running}$. (vii) Every transition's guard conjoins run ; hence from a state with $\neg run$, $Next$ is disabled and $\Box[Next]_{vars}$ permits only stuttering. (Weak fairness of the five named actions is vacuous there, since none is enabled.) $\square$

Remark 4.17 (Fail-stop precludes duplication and reordering). Combining Theorem 4.16(vi,vii) with Theorem 4.13: after a partial write failure, c and L are exactly what they were before the failed batch, the native scrollbar ends in a **FailedWrite**- or **Replay**-tagged *prefix* of immutable finalized/stable rows, and no further transition exists. There is thus no state from which a retry could re-emit rows, and no interleaving in which a younger block's rows precede an older block's in either history.

4.8 Conditional progress and liveness

Recall the weak-fairness conjuncts of Definition 3.4 and write $\rightsquigarrow$ for “leads to”. Define

$$\text{AllFin} \iff \forall i \in [\gamma] : \varphi_i \in \{\text{Finalized}, \text{Committed}\}, \quad \text{AllCom} \iff c = \gamma \wedge L = C(c).$$

Lemma 4.18 (Resize exhaustion). *In every behavior of Spec, only finitely many RESIZE steps occur; after the last one, ω and h are constant and no new replay begins.*

Proof. ν increases by one at each RESIZE and at no other step, and $\nu \leq R_{\max}$ by T. RESIZE is the only transition that sets m_r to a non-None value. $\square$

Theorem 4.19 (Replay termination). $\text{Spec} \models (\text{rep} \wedge run) \rightsquigarrow (\neg \text{rep} \vee \neg run)$.

Proof. Assume, toward contradiction, a behavior with a suffix on which $\text{rep} \wedge run$ holds forever. By Lemma 4.18 pass to a further suffix with no RESIZE steps. On this suffix m_r can only be changed by **REPLAYSYNCHRONOUSUCCESS** (to None, contradiction) and run only by a halting transition (contradiction), so suppose neither ever fires. Two cases on $prep$:

If $prep$ is false at some state of the suffix, then **PREPAREREPLAY** is enabled there ($run \wedge \text{rep} \wedge \neg prep$) and remains enabled at every subsequent state until taken: the only transitions that modify $prep$ or m_r are RESIZE (excluded), **PREPAREREPLAY** itself, and the replay writes (which require $prep$). By $\text{WF}(\text{PREPAREREPLAY})$ it is eventually taken, after which $prep$ holds.

Once $prep$ holds, the scheduler gate restricts $Next$ to the two replay writes; the action **REPLAYSYNCHRONOUSUCCESS** is then enabled ($run \wedge \text{rep} \wedge prep$) and stays enabled while neither write fires (no other transition can execute). By $\text{WF}(\text{REPLAYSYNCHRONOUSUCCESS})$, eventually the success fires (making $m_r = \text{None}$) or the failure fires first (making $\neg run$)—either contradicts the assumption. $\square$

Theorem 4.20 (Flush-driven completion). $\text{Spec} \models (\text{AllFin} \wedge flush \wedge shut \wedge run \wedge \neg \text{rep}) \rightsquigarrow (\text{AllCom} \vee \neg run)$.

Proof. Let s_0 satisfy the antecedent and consider the behavior from s_0 , assuming run persists (otherwise the consequent holds). Under $shut$ (which no transition unsets), the transitions **CREATE**, **ADMIT**, **UPDATE**, **REQUESTALLOCATION**, **APPLYALLOCATION**, both finalizations, **APPENDSTABLE**, **RESIZE**, **BEGINGRACEFULSHUTDOWN**, and **DETACHEXIT** are disabled by their $\neg shut$ guards. In particular no new replay can begin (only RESIZE begins one), so $\neg \text{rep}$ persists; γ is frozen; AllFin persists (phases only move toward **Committed**); and $flush$ persists, so req holds at every state.

Claim: while $c < \gamma$, the action $\text{RETIRESUCCESS}^\exists$ is enabled. Indeed $b = \gamma$ satisfies $b \in (c+1)..N$, the range $c+1..b$ is finalized (AllFin, minus committed prefix), req holds, and $\neg \text{rep} \wedge \text{run}$ hold. The only enabled non-stuttering transitions from such states are RETIRESUCCESS , RETIREFAILURE , $\text{COMPLETEAPPENDONLY}$, BEGINFLUSH (disabled: *flush* already), and GRACEFULEXIT (disabled while $c < \gamma$). If RETIREFAILURE fires, $\neg \text{run}$ and we are done. $\text{COMPLETEAPPENDONLY}$ strictly increases c . Since $\text{RETIRESUCCESS}^\exists$ remains enabled at every state with $c < \gamma$, weak fairness guarantees that, unless c reaches γ by $\text{COMPLETEAPPENDONLY}$ steps first, a $\text{RETIRESUCCESS}(b)$ step occurs, which sets $c' = b \geq c + 1$. Each frontier-advancing step increases c by at least one and $c \leq N$, so after at most N such steps $c = \gamma$. At that point ph fails (its conjunct $c < \gamma$), so by ECH $L = C(c) \cdot \langle \rangle = C(c)$: exactly AllCom. $\square$

Theorem 4.21 (Pressure-driven retirement). *Let $i \in \mathcal{B}$. Suppose a reachable state s satisfies*

$$\text{run} \wedge \neg \text{rep} \wedge c = i-1 \wedge \varphi_i = \text{Finalized} \wedge (\text{flush} \vee \gamma - c \geq \Lambda).$$

Then $\text{Spec} \models$ from s , eventually $c \geq i \vee \neg \text{run}$.

Proof. Assume *run* persists and $c = i-1$ persists (otherwise, since c is monotone, $c \geq i$ eventually and we are done). Then: *flush*, once true, is stable; and $\gamma - c$ is nondecreasing while c is fixed (γ is nondecreasing), so the disjunct that held at s holds forever, hence req holds forever. $\varphi_i = \text{Finalized}$ persists while $c < i$ (a finalized block changes phase only by committing, which forces $c \geq i$). By Lemma 4.18 and Theorem 4.19, eventually a suffix begins on which $\neg \text{rep}$ holds forever. On that suffix $\text{RETIRESUCCESS}^\exists$ is continuously enabled (witness $b = i$: FinalizedRange is the single block i , req holds, $\neg \text{rep} \wedge \text{run}$ hold), so by weak fairness some $\text{RETIRESUCCESS}(b)$ with $b \geq c + 1 = i$ fires—contradicting the persistence of $c = i-1$. (A RETIREFAILURE instead yields $\neg \text{run}$.) $\square$

Remark 4.22 (Sharpness: row pressure alone does not force retirement). Theorem 4.21 conditions on the *persistent* pressure disjuncts (*flush*, or count pressure $\gamma - c \geq \Lambda$, which is monotone while the frontier is fixed). Pure *row* pressure $D > h$ is not persistent and does not suffice in general: with $\Lambda = 4$, $N = 3$, a finalized head ($f_1 = \langle a \rangle$), an active mutable block 2, and a queued block 3 under $h = 1$, we have $D = 3 > 1$; a subsequent UPDATE shrinking w_2 and a Preserve resize to $h' = 3$ give $D = 3 \leq 3$, extinguishing press with $\gamma - c = 3 < \Lambda$ and no flush—after which RETIRESUCCESS is disabled forever and an infinite stutter-fair suffix keeps $c = 0$. The mechanically checked property **QueuedPressureRetirement** (§5) adds the hypothesis $\exists j : \varphi_j = \text{Queued}$ and is checked at $\Lambda = 2$; there the hypothesis *implies* persistent count pressure—a queued block j and the finalized head i are distinct uncommitted created blocks, so $\gamma - c \geq 2 = \Lambda$ —and the property is an instance of Theorem 4.21. This is the precise sense in which queued demand guarantees that older finished work is swept out of the viewport.

5 Mechanical Verification in TLa^+

The entire development of §2–§4 is a faithful mathematical reading of the module `ElasticSlots.tla`; this section records the correspondence and the exhaustive model-checking evidence.

5.1 Transition correspondence

Table 3 maps each transition to its TLa^+ action. The scheduler gate of Definition 3.4 is the IF `replayPrepared` branch of `Next`; the five weak-fairness conjuncts appear verbatim in `Spec`. Failure nondeterminism is bounded by $\text{MaxFailureRows} = K_{\max} = 2NL_{\max}$.

This paper	TLA ⁺ action	This paper	TLA ⁺ action
CREATE(m)	Create	RETIRESUCCESS(b)	RetireSuccess
ADMIT(i)	Admit	RETIREFAILURE(b, k)	RetireFailure
UPDATE(i, s)	Update	RESIZE($\omega^\dagger, h^\dagger, p, k$)	Resize
REQUESTALLOCATION($\tau^\dagger$)	RequestAllocation	PREPAREREPLAY	PrepareReplay
APPLYALLOCATION(i)	ApplyAllocation	REPLAYSYNCHRONOUSUCCESS	ReplaySynchronousSuccess
FINALIZEACTIVE(i, s)	FinalizeActive	REPLAYSYNCHRONOUSFAILURE(k)	ReplaySynchronousFailure
FINALIZEQUEUED(i, s)	FinalizeQueued	BEGINGRACEFULSHUTDOWN	BeginGracefulShutdown
APPENDSTABLE	AppendStable	GRACEFULEXIT(k)	GracefulExit
COMPLETEAPPENDONLY	CompleteAppendOnly	DETACHEXIT(k)	DetachExit
BEGINFLUSH	BeginFlush		

Table 3: Transition-to-action correspondence (TLA⁺ parameter lists—declaration, snapshot, newTarget, batchEnd, count, newWidth, newHeight, resizePolicy, pushed, push—mirror the mathematical parameters positionally).

Likewise for the derived operators: ρ_ω is Render/DoubleRows; tag/ntag are Tag/NativeTag; $C(k)$ is CommittedRows; $A(c)$ is PartialHeadRows (with ph as PartialHeadExists); $F_{\log}, F_{\text{phys}}^q$ are RetirementRows/NativeRetirementRows; pres, vis, σ are Presented, VisiblePresented, SummaryRows; $\mathcal{A}$ is AllocationStateOK; κ is CanonicalAllocation; B is BridgeHeight; dm, D are SnapshotHeight/FullRows; press, req are Pressure/RetirementRequested; Q is Screen; $\mathcal{R}$, room, cut^* are ReplayRows, ReplayRoom, RequiredReplayCut.

5.2 Checked invariants and temporal properties

Table 4 lists the ten state invariants and ten temporal properties in the model-checking configurations, with their counterparts in this paper.

5.3 Configurations and results

Three boundary configurations were checked exhaustively with TLC (version 2026.03.31, OpenJDK 17, Apple M4 Max, 16 worker threads), all with the snapshot universe $\mathbb{S} = \{\langle \rangle, \langle a \rangle, \langle a, b \rangle\}$ over $\Sigma = \{a, b\}$ (satisfying Assumption 2.1) and deadlock checking disabled—terminal states (halted hosts) are intended. No invariant violation, no property violation, and no liveness counterexample was found in any configuration.

Why these boundaries. The main configuration ($N=2, H=1$) maximizes contention: a one-row transcript forces the overflow/summary path (Theorem 4.2(iii)) whenever two blocks are presented, exercises zero-height degradation via resizes to $h=0$, and—with $\Lambda=2$ —realizes the persistent count pressure of Remark 4.22, so that QueuedPressureRetirement is checked in exactly the regime where Theorem 4.21 applies. The bridges configuration ($H=3$, no resizes) admits allocations up to 3 and therefore drives the deep-shrink trajectory $3 \rightarrow 2 \rightarrow 1$ of Lemma 4.4(iii) together with concurrent request/apply interleavings against the reservation invariant. The resize configuration ($R_{\max}=2$) covers resize-during-replay: a second resize arriving before REPLAYSYNCHRONOUSUCCESS, which preserves the replay window but resets preparation, plus all {Preserve, Append, Rebuild} $\times$ width/height combinations, epoch resets, and emulator-push capture. Failure nondeterminism (RETIREFAILURE, REPLAYSYNCHRONOUSFAILURE at every prefix length up to $K_{\max}$) and both exits are enabled in all three configurations.

TLA ⁺ name	Kind	This paper
TypeOK	invariant	T (Def. 4.1)
LifecycleShape	invariant	LS
SnapshotDiscipline	invariant	SD
EmissionDiscipline	invariant	ED
Capacity	invariant	CAP
ExactCommittedHistory	invariant	ECH, eq. (1)
NoPrematureHistory	invariant	Corollary 4.14
ScreenCapacity	invariant	Corollary 4.15; Theorem 4.2
ReplayShape	invariant	RS; Lemma 4.11(i)
NativeSourceSafety	invariant	NSS
HistoryMonotonicity	action property	Theorem 4.16(i)
NativeEpochDiscipline	action property	Theorem 4.16(ii)
FinalImmutability	action property	Theorem 4.16(iii)
AppendOnlyMonotonicity	action property	Theorem 4.16(iv)
ResizeKeepsLogicalHistory	action property	Theorem 4.16(v)
FailedWriteStops	invariance	Theorem 4.16(vi)
StoppedQuiescence	action property	Theorem 4.16(vii)
FlushLiveness	leads-to	Theorem 4.20
ReplayLiveness	leads-to	Theorem 4.19
QueuedPressureRetirement	leads-to	Theorem 4.21, Remark 4.22

Table 4: Checked properties and their counterparts.

	ElasticSlots	ElasticSlotsBridges	ElasticSlotsResize
N / H	2 / 1	1 / 3	1 / 1
$R_{\max} / \Lambda$	1 / 2	0 / 1	2 / 1
Invariants checked	all 10	7	9
Temporal properties	all 10	6	8
States generated	568,654	1,736	117,042
Distinct states	219,721	501	52,638
Graph depth	18	9	16
Liveness branches	4	1	1
Behavior-graph states (liveness)	878,884	501	52,638
Wall time	22 s	< 1 s	3 s
Fingerprint collision prob.	4.4×10^{-9}	3.4×10^{-14}	2.4×10^{-10}

Table 5: Exhaustive TLC results (August 24, 2026).

Division of labor. TLC establishes the theorems of this paper—for the checked instances—directly on the specification text, including the inductive stability arguments of Theorem 4.13 whose case analyses are most error-prone by hand (notably the interaction of RS with the scheduler gate, and NSS across frontier motion). The hand proofs above are stated for all parameter values of Assumption 2.1 and identify *why* each invariant is stable, which the model checker does not; where a checked property is parameter-sensitive we have said so explicitly (Remark 4.22). The two artifacts are kept in lock-step by the correspondence of Tables 3 and 4.

6 Conclusion

Elastic Speculative Slots reconciles speculative, concurrent, elastically animated live output with the unforgiving append-only nature of terminal scrollback by decoupling three layers—semantic blocks, an exactly-once logical ledger, and epoch-scoped physical native history—and by making every crossing between layers an explicit, provable transition. The reservation invariant makes concurrent slot animation compositional (Lemma 4.4); the exactness equation $L = C(c) \cdot A(c)$ reduces the entire logical history to two monotone quantities, the frontier and the head’s emitted prefix (Lemma 4.6, Theorem 4.13); the streaming lemma shows that every abstract history append is one cursor discipline away from a real VT (Lemma 4.8); and the prepared-frame gate turns “resize replay without tearing” from a rendering aspiration into an inductive invariant (Lemma 4.11). Failure is fail-stop by construction: a torn write leaves a tagged, immutable, attributable prefix and a frontier that never lies. Under weak fairness the protocol drains: replays terminate, flushes complete, and pressure—when persistent—sweeps finished work into history exactly once and exactly in order (Theorems 4.19–4.21). All of it is checked, state by state, against `ElasticSlots.tla`.